

Processing Entropy as a Metric Space

Distance Between Any Two Ways of Doing the Same Thing

Registry: [\[HOWL-MATH-17-2026\]](#)

Series Path: [\[HOWL-INFO-11-2026\]](#) → [\[HOWL-INFO-12-2026\]](#) → [\[HOWL-INFO-13-2026\]](#) → [\[HOWL-MATH-14-2026\]](#) → [\[HOWL-MATH-15-2026\]](#) → [\[HOWL-MATH-16-2026\]](#) → [\[HOWL-MATH-17-2026\]](#)

Date: June 2026

DOI: 10.5281/zenodo.20630189

Domain: Information Processing Theory / Applied Mathematics

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

1. The Question

Two physicians look at the same chest X-ray. One reaches a diagnosis in three ops — a glance, a pattern match, a conclusion. The other requires twenty-eight ops — systematic review of each lung field, comparison against mental templates, elimination of differential candidates, tentative conclusion, recheck. Both arrive at the same diagnosis. The X-ray is the same. The goal is the same. The context is the same. The cost is different by nearly an order of magnitude.

Now hold the processor constant and vary the task. The experienced physician processes a classic pneumonia presentation at three ops and an unusual autoimmune presentation at twenty-two ops. Same physician, same goal — diagnose the patient — but the cost landscape across different presentations defines a profile. Some tasks are dissolved to zero. Some require full pipeline engagement. Some are outside the physician's domain entirely. The shape of that cost landscape is this physician's expertise, made visible.

These observations raise a precise mathematical question. The cost of processing — the op count a specific processor requires for a specific task — is a measurable quantity. When you collect these costs across a set of tasks for one processor, you get a profile. When you collect profiles across multiple processors, you get a space. Does that space have geometric structure? Can you define a meaningful distance between two profiles such that the distance satisfies the axioms of a metric space?

If so, expertise has a geometry. Skill gaps are distances with direction. Learning is a trajectory through a space. Clusters of similar practitioners are formally identifiable objects rather than informal impressions. The question of how far apart two processors are — in skill, in specialization, in development — becomes a question with a numerical answer.

The vocabulary is small and builds in order. Processing is what any system does when it must act on information — a CPU scheduling processes, a surgeon operating, a pilot navigating, a developer debugging. The unit of processing is the **op**: one irreducible transformation by one processor. A diagnostic question, a mirror glance, a cache lookup, a line of code read and understood — each is one op. Processing entropy is the op count a specific processor requires for a specific task. It is receiver-dependent: the same chest X-ray has processing entropy of three at the experienced physician and twenty-eight at the junior resident. When processing entropy reaches zero, the task is **dissolved** — the processor handles it structurally, without consuming its scarce sequential pipeline. The processor's total capacity is bounded by one inequality: total ops multiplied by average op duration must not exceed the available time budget. Dissolved tasks don't count against the budget. That is what makes expertise powerful — the expert's budget is free because most routine processing has dissolved.

This paper takes processing entropy as its primitive and investigates the mathematical structure that emerges when you consider many processors and many tasks simultaneously.

2. The Processing Entropy Profile

Fix a goal and a context. Let $T = \{t_1, t_2, \dots, t_m\}$ be a **task set**: a finite collection of elements that a processor in this domain might need to handle. For a physician, T might be a set of clinical presentations. For a developer, a set of bug types on a specific codebase. For a CPU, a set of memory access patterns. For a pilot, a set of flight scenarios.

For processor p , the **processing entropy profile** is the vector:

$$\mathbf{H}(p) = (Hp(t_1), Hp(t_2), \dots, Hp(t_m))$$

where $Hp(t_i)$ is the op count processor p requires for task t_i . Each component is non-negative. Zero means the task is dissolved. A positive value means the task requires active processing at that cost. There is a third possibility: the task may be outside the processor's domain entirely — a pediatrician encountering an avionics failure, a CPU encountering a philosophical argument. In this case, processing entropy is not zero (zero means dissolved, which means handled structurally) and not merely high (high means expensive but possible). It is undefined. The processor cannot perform the task at any op count.

The distinction between zero, positive, and undefined is categorical, not a matter of degree. Zero: the processor handles it without pipeline cost. Positive: the processor handles it at measurable cost. Undefined: the processor cannot handle it. Conflating zero with undefined confuses expertise (dissolved) with impossibility (outside domain). Conflating high-but-defined with undefined confuses difficulty with impossibility. The profile must preserve all three categories.

The profile vector $\mathbf{H}(p)$ is a point in m -dimensional space. Each axis corresponds to a task. The coordinate along that axis is the processing cost. A processor who has dissolved every

task in the set sits at the origin. A first-encounter novice sits far from the origin along every axis — every task is expensive. A specialist sits near the origin along some axes (their specialty, dissolved) and far from it along others (outside their specialty, expensive or undefined).

The profile makes expertise visible as geometry. Two physicians with identical profiles are interchangeable on the task set — every task costs them the same number of ops. Two physicians with different profiles have different expertise, and the shape of the difference tells you precisely how they differ: which tasks one finds cheap that the other finds expensive, which tasks both find hard, which tasks separate them most.

3. Shared Domain and Comparable Profiles

Before measuring distance between two profiles, a structural issue requires resolution. When task t_i is outside processor p 's domain, $H_p(t_i)$ is undefined. Two processors may have different domains — different subsets of the task set on which their processing entropy is defined. Comparing them requires care.

Define the **shared domain** for processors p and q :

$$T(p,q) = \{ t \in T : H_p(t) \text{ is defined} \wedge H_q(t) \text{ is defined} \}$$

This is the set of tasks that both processors can perform — where both have defined processing entropy, whether zero, low, or high. The shared domain is where comparison is meaningful. For a task outside one processor's domain, there is no basis for comparison — the processor doesn't fail expensively at the task, it simply cannot engage with it.

The size of the shared domain relative to the full task set is itself informative. Two processors with $T(p,q) = T$ share complete domain overlap — every task one can do, the other can too. They may differ enormously in cost but operate on the same territory. Two processors with $T(p,q)$ much smaller than T are in different fields — most of what one does, the other cannot do at all. The shared domain fraction is a coarse measure of domain overlap before any cost comparison begins.

The shared domain creates a complication for metric space structure. Distance between p and q is computed over $T(p,q)$. Distance between q and r is computed over $T(q,r)$. Distance between p and r is computed over $T(p,r)$. These three sets may differ. The triangle inequality — $d(p,r) \leq d(p,q) + d(q,r)$ — requires all three distances to be commensurable. If the distances are computed over different task subsets, commensurability is not automatic.

Two approaches resolve this. The **common-ground approach** restricts all comparisons in a given analysis to the universal shared domain $T^* = \cap T(p_i, p_j)$ across all processor pairs under consideration. Within T^* , all distances are computed over the same task set and all metric axioms hold automatically. The cost is information loss — tasks outside the universal shared domain are excluded even if they discriminate between some processor pairs.

The **pairwise approach** computes distance over $T(p,q)$ for each pair, normalized by shared domain size. This preserves all available information but requires proving that the normalized distance satisfies the triangle inequality, which it does only under specific conditions. When the shared domains are nested or when undefined tasks are rare relative to the task set, the pairwise approach works. When domains diverge substantially, it can fail and the common-ground approach is necessary.

For the remainder of this paper, unless stated otherwise, assume a task set T over which all processors under consideration have defined processing entropy. This is the common-ground approach. It applies naturally when comparing processors within a single domain — all physicians on a set of common clinical presentations, all developers on a shared codebase, all pilots on a standard scenario set. The pairwise extension is noted where it adds insight.

4. Distance Between Processors

With profiles defined as vectors in m -dimensional task space over a common task set, distance between processors is distance between their profile vectors. Four candidate distance functions each capture a different notion of similarity.

Euclidean distance (L^2):

$$d_2(p, q) = \sqrt{\sum_i (Hp(t_i) - Hq(t_i))^2}$$

The straight-line distance between profile points. Sensitive to large differences on individual tasks — a single task where one processor costs 30 ops and the other costs zero contributes 900 to the sum under the radical, dominating many small differences. Two processors who differ moderately across many tasks may be closer than two who differ enormously on one task. This metric captures overall profile similarity with emphasis on outlier differences.

Manhattan distance (L^1):

$$d_1(p, q) = \sum_i |Hp(t_i) - Hq(t_i)|$$

The sum of absolute per-task differences. Every op of difference contributes equally regardless of which task it belongs to. Has a direct physical interpretation: the total number of ops by which the two processors differ across the entire task set. If $d_1(p,q) = 150$, the two processors collectively differ by 150 ops of processing cost distributed across all tasks.

Chebyshev distance (L^∞):

$$d_\infty(p, q) = \max_i |Hp(t_i) - Hq(t_i)|$$

The maximum single-task difference. Two processors are as far apart as their single worst disagreement. A processor who matches the reference on every task except one, where they differ by 40 ops, has Chebyshev distance 40 regardless of how perfect the match

is elsewhere. This captures the idea of a bottleneck skill gap — the one task that most separates the processors.

Weighted distance:

$$d_w(p, q) = \sqrt{(\sum_i w_i (Hp(t_i) - Hq(t_i))^2)}$$

where w_i reflects the operational importance of task t_i — its frequency in practice, the consequence of poor performance, or both. This metric makes distance operationally meaningful rather than abstractly mathematical. Two processors who differ on a rare task are closer than two who differ on a task they perform twenty times daily.

All four satisfy the metric space axioms over a common task set. **Identity of indiscernibles:** $d(p, q) = 0$ if and only if $Hp(t_i) = Hq(t_i)$ for all i — the profiles are identical, meaning the processors have identical cost for every task. **Symmetry:** $d(p, q) = d(q, p)$ — the distance from p to q equals the distance from q to p , since absolute differences and squares are symmetric. **Triangle inequality:** $d(p, r) \leq d(p, q) + d(q, r)$ — this holds for all L_p norms and their weighted variants by established results in functional analysis.

The processing entropy profile space is therefore a metric space under any of these distances. The choice of distance determines what “close” means, which determines what clusters emerge and what trajectories optimize. Euclidean for overall similarity. Manhattan for total cost difference. Chebyshev for worst-case gap. Weighted for operational relevance. Each tells a different true story about the same pair of processors.

5. Distance Between Tasks

The profile space has a dual. Instead of holding the task set constant and comparing processors, hold the processor set constant and compare tasks.

Let $P = \{p_1, p_2, \dots, p_n\}$ be a set of processors. For task t , the **cost column** across processors is:

$$\mathbf{H}(t) = (Hp_1(t), Hp_2(t), \dots, Hp_n(t))$$

This is a vector in n -dimensional processor space. Each axis is a processor, each coordinate is that processor’s cost for this task. Two tasks are close when every processor finds them similarly costly. Two tasks are far when processors differ significantly in their relative costs for the two tasks.

Define task distance using the same families:

$$d_2(t_i, t_j) = \sqrt{(\sum_k (Hp_k(t_i) - Hp_k(t_j))^2)}$$

with Manhattan, Chebyshev, and weighted variants defined analogously. The metric axioms hold by the same arguments — the mathematics is identical with the axes transposed.

Task distance reveals structure in the domain that is invisible from any single processor’s perspective. Tasks that cluster together are tasks that all processors find similarly costly — they share some structural property that makes them equivalently difficult regardless

of who attempts them. These clusters are natural difficulty classes. They may or may not align with conventional domain taxonomies. Medical textbooks group diseases by organ system. Task distance might group them by diagnostic pattern — diseases that present similarly and require similar processing chains cluster together regardless of which organ they affect. The textbook taxonomy reflects biology. The task distance topology reflects how processors actually experience the domain.

A task that is far from all clusters is a task where processors diverge widely in cost — some find it cheap, others find it expensive. This is a **discriminating task**: one that separates skill levels. Define the **discrimination power** of a task as the variance of its cost column:

$$\text{disc}(t) = \text{var}(Hp_1(t), Hp_2(t), \dots, Hp_n(t))$$

High discrimination power means the task effectively distinguishes between processors. A task where every processor has approximately the same cost — either universally easy (all near zero) or universally hard (all high) — has low discrimination power. It tells you nothing about who is more skilled.

The set of high-discrimination tasks is the minimal assessment battery — the smallest set of tasks that, if you measured processing entropy on them alone, would maximally separate the processor population into its natural skill clusters. This is the formal version of designing a competency exam.

6. The Processing Entropy Matrix

Both views — processor profiles and task columns — are slices of a single object. The **processing entropy matrix** H has processors as rows and tasks as columns:

$$H[i,j] = Hp_i(t_j)$$

Each entry is the op count for processor i on task j . Rows are processor profiles. Columns are task cost vectors. Row distance is processor similarity. Column distance is task similarity. The matrix contains both simultaneously.

This matrix has a structural property worth investigating: its rank. The rank of H determines how many independent dimensions of variation exist in the profile space. If the matrix has rank k much less than either n (number of processors) or m (number of tasks), then the profile space has an effective dimensionality of k — processors differ along only k independent skill dimensions, even though the task set has m elements.

Low rank should be expected when processors share training tradition. Medical residents in the same program encounter similar cases in similar sequence. They dissolve similar tasks at similar rates because they are exposed to similar stimuli. Their profiles are correlated — knowing one resident's cost on task A predicts their cost on task B because A and B were encountered in the same clinical rotation. The correlation structure reduces the effective degrees of freedom.

When H is approximately low-rank, it admits a factorization:

$$\mathbf{H} \approx \mathbf{U} \times \mathbf{V}$$

where $\mathbf{U}$ is an $n \times k$ matrix (processors by skill factors) and $\mathbf{V}$ is a $k \times m$ matrix (skill factors by tasks). Each row of $\mathbf{U}$ is a processor's position in k -dimensional skill factor space. Each column of $\mathbf{V}$ is a task's loading on the k skill factors.

The **skill factors** — the columns of $\mathbf{U}$ — are the underlying dimensions of expertise. They might correspond to recognizable capabilities: clinical reasoning speed, pattern recognition breadth, procedural fluency, communication efficiency. Or they might correspond to dimensions that have no conventional name but emerge from the data as the directions along which processors most vary. Either way, they reduce the profile space from m dimensions (one per task) to k dimensions (one per skill factor), making the geometry tractable.

Each processor is now described by k coordinates instead of m . Their position in skill factor space is a compressed representation of their full profile. Distance in skill factor space approximates distance in full profile space — two processors close in skill factor space are close in full profile space — but at lower computational cost and with the interpretive benefit that each dimension has meaning.

The factorization reveals something else: which tasks load on which factors. A task with high loading on factor 1 and low loading on all others is a pure measure of factor 1. A task with significant loading on multiple factors is a compound task requiring multiple skill dimensions. The loading matrix $\mathbf{V}$ tells you what each task measures about a processor — not by domain convention but by the empirical structure of how costs vary across processors.

7. Trajectories

A processor's profile changes over time. Each day of practice, some dissolution progresses, some new tasks are encountered, and occasionally a cascade temporarily promotes dissolved tasks back to active processing. The profile vector traces a path through the space:

$$\mathbf{H}(\mathbf{p}, t_0), \mathbf{H}(\mathbf{p}, t_1), \dots, \mathbf{H}(\mathbf{p}, t_k)$$

This path is the processor's **trajectory** — the complete history of their expertise development as a curve through profile space.

Trajectories have characteristic properties. Under consistent context with ongoing practice, components of the profile vector decrease toward zero — tasks dissolve and their processing entropy drops. The trajectory moves generally toward the origin. But the motion is not uniform across components. Some tasks dissolve faster than others. The trajectory moves toward the origin along some axes quickly and along others slowly, producing a curved path rather than a straight line from starting point to origin.

Define the **dissolution velocity** at time t :

$$\mathbf{v}(\mathbf{p}, t) = d\mathbf{H}(\mathbf{p})/dt$$

This is a vector in task space. Each component is the rate of change of processing entropy for one task. Negative components indicate dissolution in progress — the task is getting cheaper. Zero components indicate either full dissolution (already at zero, no further change) or stalled dissolution (plateau, no progress). Positive components indicate regression — a cascade has promoted a dissolved task back to active processing, or context change has increased the cost.

The magnitude of the velocity vector is the total rate of skill change:

$|\mathbf{v}(\mathbf{p}, t)| = \sqrt{(\sum_i (dH_p(t_i)/dt)^2)}$ High magnitude means rapid change in the profile — either rapid improvement (dissolving many tasks quickly) or rapid degradation (cascade promoting many tasks). The direction of the velocity vector indicates where change is concentrated.

Define **trajectory efficiency** as the ratio of displacement to path length:

$$\eta = |\mathbf{H}(\mathbf{p}, t_k) - \mathbf{H}(\mathbf{p}, t_0)| / \sum_j |\mathbf{H}(\mathbf{p}, t_{j+1}) - \mathbf{H}(\mathbf{p}, t_j)|$$

Efficiency near one means the processor moved directly from start to current position without detours — steady dissolution without cascades or backtracking. Efficiency below one means the path included reversals, oscillations, or indirect routing through the space. A trajectory with multiple cascades has low efficiency — each cascade pushes the profile away from the origin temporarily, adding path length without net displacement.

Trajectories across a cohort of processors in the same training program reveal the structure of expertise development. If all trajectories follow approximately the same path — dissolving the same tasks in the same order — there is a **characteristic curriculum**: a natural sequence of dissolution that the training program (or the domain's structure) imposes. The characteristic path is the average trajectory across the cohort, and deviation from it measures how idiosyncratic a given processor's development is.

If trajectories diverge — different processors dissolving different tasks first, arriving at expertise by different routes — then the path through profile space is not determined by the domain but by individual differences in training exposure, aptitude, or strategy. Multiple efficient paths to the same destination suggest that training programs have genuine choices about sequencing. A path that is consistently shorter (less total path length from start to expert region) represents a more efficient curriculum regardless of which tasks it dissolves first.

8. Skill Gap as Geometric Object

The informal phrase “skill gap” acquires a precise geometric definition. The gap between processor $\mathbf{p}$ and a reference processor $\mathbf{r}$ is the vector:

$$\mathbf{g}(\mathbf{p}, \mathbf{r}) = \mathbf{H}(\mathbf{p}) - \mathbf{H}(\mathbf{r})$$

Each component $g_i = H_p(t_i) - H_r(t_i)$ is the per-task cost difference. The reference $\mathbf{r}$ may be an expert (measuring how far $\mathbf{p}$ is from expertise), a standard (measuring how far $\mathbf{p}$ is from certification level), or another specific processor (measuring how $\mathbf{p}$ and $\mathbf{r}$ differ).

The gap is a vector, not a scalar. It has magnitude and direction. The magnitude is the distance between p and r under whichever metric is chosen — the total size of the gap. The direction tells you which tasks contribute most — where the gap lives in task space.

Decompose the gap vector by sorting components from largest to smallest to produce the **gap profile**: the shape of the distance between two processors. Two gap profiles with the same magnitude can have very different shapes:

A **concentrated gap** has a few large components and many near-zero components. The processor matches the reference on most tasks but diverges sharply on a handful. The training prescription is targeted: dissolve these specific tasks. The gap profile has a spike.

A **distributed gap** has many moderate components and no extreme ones. The processor is moderately more expensive than the reference on everything. The training prescription is general: more overall experience. The gap profile is flat.

A **mixed gap** has some large components and some moderate ones. The training prescription has both targeted and general elements.

The gap profile shape determines training strategy more precisely than the gap magnitude alone. Two processors equidistant from the reference (same magnitude) may need entirely different training programs because their gap profiles have different shapes — one needs targeted skill building, the other needs broad experience.

Define the **gap closure rate** as the rate at which gap magnitude decreases:

$$\rho(p, r, t) = -d|g(p, r)|/dt$$

Positive closure rate means the processor is approaching the reference — the gap is shrinking. Negative closure rate means the processor is falling behind — the reference is dissolving faster or the processor is experiencing cascades. Zero closure rate means the gap is stable — neither closing nor widening.

The closure rate decomposes by task. The rate at which each component of the gap closes is:

$$\rho_i(p, r, t) = -d(Hp(t_i) - Hr(t_i))/dt$$

When the reference is static (an expert whose profile is no longer changing), this simplifies to the processor's dissolution rate per task. When the reference is also developing (a peer, or a moving standard), the closure rate depends on the differential dissolution rates — the processor must dissolve faster than the reference to close the gap.

The gap vector, the gap profile, and the closure rate together give a complete geometric description of one processor's position relative to another. Distance tells you how far. Direction tells you which tasks. Shape tells you whether the gap is concentrated or distributed. Closure rate tells you whether it's shrinking. This is what "skill gap analysis" means when it has a mathematical foundation.

9. Clusters

With distance defined, the space of processor profiles has explorable structure. Processors that are close to each other and far from others form clusters — groups whose members are more similar to each other than to outsiders.

Skill level clusters emerge when the dominant variation in the profile space is distance from the origin. Novice processors cluster in a region far from the origin (high cost on most tasks). Intermediate processors cluster closer. Experts cluster near the origin. The clusters form concentric shells at different distances from the origin, corresponding to different amounts of total dissolution.

The gaps between shells are transition zones — regions of profile space that processors pass through relatively quickly during development. A large gap between the novice and intermediate clusters means the transition from novice to intermediate involves rapid dissolution of many tasks in a short period — a phase transition in skill development. A small gap means the transition is gradual.

Specialization clusters emerge when the dominant variation is direction from the origin rather than distance. Two specialists equidistant from the origin but in different directions have dissolved different task subsets. A cardiac surgeon and a neurosurgeon have similar total dissolution (comparable distance from origin) but different profiles (different directions). They cluster by specialty, not by skill level.

In practice, both effects operate simultaneously. The profile space has radial structure (skill level, distance from origin) and angular structure (specialization, direction from origin). The full clustering reveals both: concentric shells of skill level subdivided into wedges of specialization.

Boundary regions between clusters correspond to transitions — in skill level (between shells), in specialization (between wedges), or both (diagonal transitions where a processor is changing both skill level and specialty). Sparsely populated boundaries are natural transition points that processors pass through quickly. Densely populated boundaries are decision points where processors may plateau before committing to a direction.

Outliers — processors far from all clusters — are diagnostically interesting. A processor near the origin but far from the expert cluster has dissolved an unusual combination of tasks. A processor far from the origin but isolated from the novice cluster has an unusual cost profile — perhaps high cost on typically easy tasks, suggesting regression or an unusual background. Outlier detection in the metric space is a formal method for identifying processors who need attention — either exceptional processors to learn from or struggling processors to support.

10. Task Topology

Distance between tasks induces structure on the task set itself. Tasks that are close (all processors find them similarly costly) form neighborhoods. The collection of neighborhoods

and their relationships is the **task topology** — the shape of the domain as processors experience it.

Construct the **task graph**: tasks are nodes, and two tasks are connected by an edge if their distance is below a threshold. Edge weight is inverse distance — closer tasks have stronger connections. Connected components of the task graph are natural task clusters from the processor’s perspective.

These clusters may not match conventional domain categories. Conventional categories are defined by subject matter — diseases by organ system, bugs by subsystem, flight scenarios by weather type. Task clusters defined by processing entropy distance are defined by what it costs to do them — tasks that require similar processing chains cluster together regardless of their subject-matter category.

Consider medicine. A conventional taxonomy separates cardiac disease from pulmonary disease from neurological disease. But from the processing entropy perspective, a straightforward cardiac presentation and a straightforward pulmonary presentation may cluster together (both cost experienced physicians three ops — the pattern recognition is structurally similar) while an unusual cardiac presentation clusters with unusual presentations of any organ system (all cost twenty-plus ops — the diagnostic reasoning chain is structurally similar regardless of which organ is involved). The task topology reflects processing structure, not biological structure.

This disconnect between conventional taxonomy and processing topology has practical consequences. Training programs organized by conventional categories (a cardiac rotation, a pulmonary rotation) may not be optimally organized for dissolution efficiency. If tasks cluster by processing structure rather than organ system, a training program organized by processing pattern (straightforward presentations across all organ systems, then atypical presentations across all organ systems) might produce faster dissolution because structurally similar tasks enable transfer — dissolving one accelerates dissolution of others in the same processing-topology cluster.

Define **transfer affinity** between tasks t_i and t_j : the degree to which dissolution of t_i reduces the first-encounter cost of t_j . If dissolving the pattern-recognition chain for classic pneumonia reduces the cost of recognizing classic heart failure (because the pattern-recognition structure is similar even though the organs differ), the transfer affinity is high. If dissolving pneumonia recognition has no effect on recognizing heart failure, the transfer affinity is zero.

Transfer affinity may differ from processing entropy distance. Two tasks may have similar cost profiles across processors (small distance) without transferring to each other — they are independently easy or independently hard for structural reasons unrelated to shared processing chains. Conversely, two tasks with different cost profiles might have high transfer affinity — dissolving one changes the other’s cost precisely because the tasks are currently at different difficulty levels but share underlying structure.

The transfer affinity graph — tasks as nodes, edges weighted by transfer affinity — is the map of where dissolution investment has leverage. A task with high transfer affinity to many other tasks is a **high-leverage task**: dissolving it accelerates dissolution across a

neighborhood of the task space. Identifying high-leverage tasks and prioritizing them in training is a formal optimization of training sequence.

11. Computational Test Case

Every formal object defined in this paper is measurable in existing systems. This section grounds the formalism in two domains where measurement is precise.

Software engineering. The task set is a collection of bug types on a specific codebase — null pointer dereference, off-by-one error, race condition, memory leak, API misuse, configuration error, and so on, as fine-grained as the codebase’s bug taxonomy allows. The processor set is a development team. Processing entropy for each developer-bug pair is measurable from existing instrumentation: IDE telemetry (keystrokes, file navigation, tool invocations), version control timestamps (time from branch creation to fix commit), and screen recording (direct op counting). A developer who reads one log line, jumps to the relevant function, and commits a fix in three minutes has low processing entropy for that bug type. A developer who searches six files, reads documentation, tries two failed approaches, and fixes the bug in ninety minutes has high processing entropy for the same bug type.

Build the processing entropy matrix from observed data over a development cycle. Rows are developers. Columns are bug types. Each cell is the average op count (or time, as a proxy) for that developer on that bug type. Missing cells (bug types a developer has never encountered) are undefined.

Compute processor distances. Developers who are close across the task set are interchangeable on this codebase. Developers who are far apart have different expertise profiles — one may be strong on concurrency bugs and weak on API issues, the other the reverse. The distance matrix reveals pairing opportunities (pair a developer with a far-apart partner to maximize knowledge transfer) and staffing risks (if two close developers leave, their replacements cover the same profile, leaving gaps where neither was strong).

Compute task distances. Bug types that cluster together are bugs that all developers find similarly difficult. These clusters reveal the codebase’s natural difficulty structure — which problems are universally easy (everyone has dissolved the relevant debugging chains), which are universally hard (no one has dissolved them), and which discriminate (some developers have dissolved them and others haven’t).

Track trajectories over months. A new team member’s profile starts far from the origin and moves toward it as they dissolve debugging chains for this codebase. The trajectory reveals which bug types they are dissolving fastest (steepest descent along those axes) and which remain expensive (flat along those axes). The trajectory compared to the team’s characteristic path reveals whether the new member is following the typical onboarding progression or taking an unusual route.

Measure gap closure rates against the team median profile. A new member closing the gap at a steady rate is on track. A member whose closure rate has dropped to zero has

plateaued — they are no longer dissolving new debugging chains and their profile has stalled in the space. A member whose gap is widening has experienced a regression — perhaps a codebase refactor invalidated their dissolved navigation patterns, producing a cascade in the profile space.

Computation. The task set is a collection of memory access patterns: sequential scan, strided access at various intervals, random access with various locality characteristics, pointer-chasing with various chain lengths. The processor set is a collection of cache configurations — different associativities, capacities, replacement policies, and prefetch strategies. Processing entropy for each configuration-pattern pair is the average memory access cost in cycles, directly measurable with hardware performance counters.

The processing entropy matrix has cache configurations as rows and access patterns as columns. Row distance tells you which configurations behave similarly across the pattern set — configurations that are close are interchangeable for this workload mix. Column distance tells you which access patterns are similar from the cache’s perspective — patterns that cluster together have similar cost profiles regardless of cache configuration.

The rank of this matrix reveals how many independent dimensions of cache behavior exist across the pattern set. If the matrix has low rank, a small number of cache characteristics (capacity tier, associativity class, prefetch aggressiveness) explain most of the variation, and knowing a configuration’s performance on a few representative patterns predicts its performance on the rest.

Transfer affinity between access patterns corresponds to prefetch effectiveness. Patterns with high transfer affinity benefit from the same prefetch strategy — dissolving the access cost for one pattern (by caching or prefetching) simultaneously reduces the cost for the other. The transfer affinity graph maps which access pattern optimizations have collateral benefit.

Both test cases demonstrate that the formal objects — profiles, distances, matrices, trajectories, gaps, clusters, task topology — are not abstract constructions awaiting future data but structures already implicit in existing measurements, requiring only the framework to make them visible and interpretable.

12. Scope and Open Problems

This paper establishes the processing entropy profile as a well-defined point in a metric space, verifies that four natural distance functions satisfy the metric axioms, defines the dual task-distance metric, constructs the processing entropy matrix as the unifying object, introduces trajectories as paths through profile space, gives skill gap a geometric definition as a vector with magnitude and direction, identifies cluster structure as emergent from the metric, and defines the task topology as the shape of a domain experienced by processors.

The following remain open.

Effective dimensionality. Is the rank of the processing entropy matrix stable within a domain — does the number of underlying skill factors remain constant as you add

more processors or more tasks? If so, the dimensionality is a property of the domain, not the sample. If not, dimensionality depends on population characteristics and task selection, complicating cross-study comparison. Determining which case holds, and for which domains, requires large-scale empirical matrix construction and rank analysis.

Optimal assessment design. Given a population of processors, what minimal subset of tasks maximally discriminates between skill levels? This is a formal version of designing a competency examination. The discrimination power statistic defined in Section 5 identifies individual high-discrimination tasks, but the optimal subset may not be the top-k individual discriminators — tasks may be redundant (high discrimination but measuring the same skill factor) or complementary (moderate individual discrimination but jointly covering distinct factors). The optimization over task subsets, accounting for redundancy and complementarity, is a combinatorial problem with structure that the matrix factorization may make tractable.

Trajectory prediction. Given a processor’s current position and velocity in profile space, can future position be predicted? If dissolution curves follow a known family (power law, exponential), each component of the trajectory is a curve from that family, and the trajectory is predictable from current position plus velocity plus the curve parameters. The accuracy of prediction depends on context consistency — in stable conditions, trajectories are smooth and predictable; with cascades, trajectories are disrupted and prediction requires cascade modeling from dissolution geometry as described in prior work. The boundary between predictable and unpredictable trajectory segments is itself an open question.

Transfer affinity formalization. This paper defines transfer affinity between tasks but does not derive it from more primitive quantities. Under what conditions does small processing entropy distance imply high transfer affinity? The relationship is intuitive (tasks that cost the same should share structure that enables transfer) but not logically necessary (tasks may independently happen to have similar costs without sharing processing chain structure). A formal derivation of transfer affinity from processing chain analysis — showing that tasks whose reduction chains share sub-chains have high transfer affinity — would ground the concept in the framework’s existing primitives.

Dynamic task sets. The task set T is assumed fixed throughout this paper. In practice, tasks change. New bug types appear as a codebase evolves. New diseases emerge. New flight scenarios arise from new aircraft capabilities. When the task set changes, the profile space changes dimensionality — new axes appear, old ones may become irrelevant. How the metric space transforms under task set evolution, and whether distance relationships are preserved across task set changes, is an open structural question with practical implications for longitudinal skill tracking.

Cross-domain comparison. Can profiles from different domains be meaningfully compared if the domains share structural similarity? A physician’s diagnostic profile and a mechanic’s troubleshooting profile both describe processing entropy over reduction chains — can the distance between them be computed if the tasks are structurally mapped? The processing entropy matrix factorization suggests a path: if the skill factors in two domains are structurally analogous (both have a “pattern recognition” factor and a “procedural ex-

ecution” factor), cross-domain distance might be defined in skill factor space rather than task space. Whether such structural analogy holds empirically is an open question.

Each of these open problems has sufficient structure from this paper to be a well-defined investigation. The metric space, the matrix, the trajectory, and the task topology provide the formal coordinates within which the investigations can proceed.

Appendix: Supporting Tables

HOWL-MATH-17-2026

Table A: Formal Definitions

Symbol	Name	Definition	Unit	Space
T	Task set	Finite collection $\{t_1, t_2, \dots, t_m\}$ of elements a processor in this domain might handle	—	Domain-specific
P	Processor set	Finite collection $\{p_1, p_2, \dots, p_n\}$ of processors under comparison	—	Domain-specific
$Hp(t_i)$	Processing entropy	Op count processor p requires for task t_i ; zero if dissolved; undefined if outside domain	ops	Non-negative reals $\cup \{\text{undefined}\}$
$\mathbf{H}(p)$	Processing entropy profile	Vector $(Hp(t_1), \dots, Hp(t_m))$ for processor p across task set T	ops per component	$\mathbb{R} \geq 0^m$ (with possible undefined components)
$\mathbf{H}(t)$	Task cost column	Vector $(Hp_1(t), \dots, Hp_n(t))$ for task t across processor set P	ops per component	$\mathbb{R} \geq 0^n$ (with possible undefined components)
H	Processing entropy matrix	Matrix with $H[i,j] = Hp_i(t_j)$; processors as rows, tasks as columns	ops per cell	$\mathbb{R} \geq 0^{n \times m}$ (with possible undefined cells)

Symbol	Name	Definition	Unit	Space
$T(p,q)$	Shared domain	$\{ t \in T : H_p(t) \text{ defined} \wedge H_q(t) \text{ defined} \};$ tasks both processors can perform	—	Subset of T
T^*	Universal shared domain	$\cap_{i,j} T(p_i, p_j)$ across all processor pairs; tasks all processors can perform	—	Subset of T
$d_2(p,q)$	Euclidean distance	$\sqrt{(\sum_i (H_p(t_i) - H_q(t_i))^2)}$ over shared domain	ops	$\mathbb{R} \geq 0$
$d_1(p,q)$	Manhattan distance	$\sum_i H_p(t_i) - H_q(t_i) $ over shared domain	ops	$\mathbb{R} \geq 0$
$d_\infty(p,q)$	Chebyshev distance	$\max_i H_p(t_i) - H_q(t_i) $ over shared domain	ops	$\mathbb{R} \geq 0$
$d_w(p,q)$	Weighted distance	$\sqrt{(\sum_i w_i (H_p(t_i) - H_q(t_i))^2)}$ over shared domain	ops (weighted)	$\mathbb{R} \geq 0$
$\text{disc}(t)$	Discrimination power	$\text{var}(H_{p_1}(t), \dots, H_{p_n}(t));$ variance of cost column for task t	ops ²	$\mathbb{R} \geq 0$
$\mathbf{g}(p,r)$	Skill gap vector	$\mathbf{H}(p) - \mathbf{H}(r);$ per-task cost difference from reference	ops per component	$\mathbb{R}^m$ (signed)
$\rho(p,r,t)$	Gap closure rate	$-\text{d} \mathbf{g}(p,r) /\text{d}t;$ rate of gap magnitude decrease	ops/time	$\mathbb{R}$ (signed)
$\mathbf{v}(p,t)$	Dissolution velocity	$\text{d}\mathbf{H}(p)/\text{d}t;$ rate of profile change	ops/time per component	$\mathbb{R}^m$ (signed)
η	Trajectory efficiency	Displacement / path length; ratio of net progress to total movement	dimensionless, $\mathbb{R} [0,1]$	
$U \times V$	Matrix factorization	$H \approx U(n \times k) \times V(k \times m);$ U = processor positions in skill factor space; V = task loadings on factors	ops	$\mathbb{R}^{n \times k} \times \mathbb{R}^{k \times m}$
k	Effective dimensionality	Rank of H or number of retained factors in approximate factorization	count	$\mathbb{N}$

Symbol	Name	Definition	Unit	Space
$\text{aff}(t_i, t_j)$	Transfer affinity	Degree to which dissolution of t_i reduces first-encounter cost of t_j	ops reduced	$\mathbb{R} \geq 0$

Table B: Metric Axiom Verification

Axiom	Statement	L^2 (Euclidean)	L^1 (Manhattan)	L^∞ (Chebyshev)	Weighted L^2	Notes
Non-negativity	$d(p, q) \geq 0$	✓ Squares and square root non-negative	✓ Absolute values non-negative	✓ Maximum of non-negatives is non-negative	✓ Weights positive, rest follows from L^2	Holds trivially for all L_p norms
Identity of indiscernibles	$d(p, q) = 0$ iff $\mathbf{H}(p) = \mathbf{H}(q)$	✓ Sum of squares is zero iff each term is zero	✓ Sum of absolutes is zero iff each term is zero	✓ Maximum is zero iff each term is zero	✓ With positive weights, sum is zero iff each term is zero	“Identical profiles” means identical cost on every shared task

Axiom	Statement	L^2 (Euclidean)	L^1 (Manhattan)	L^∞ (Chebyshev)	Weighted L^2	Notes
Symmetry	$d(p,q) = d(q,p)$	$\checkmark (a-b)^2 = (b-a)^2$	$\checkmark a-b = b-a $	$\checkmark a-b = b-a $	$\checkmark$ Same as L^2	Distance from junior to senior equals distance from senior to junior
Triangle inequality	$d(p,r) \leq d(p,q) + d(q,r)$	$\checkmark$ Minkowski inequality for $p=2$	$\checkmark$ Minkowski inequality for $p=1$	$\checkmark$ Minkowski inequality for $p=\infty$	$\checkmark$ Minkowski with weighted inner product	Holds over common task set T^* ; requires care when shared domains differ

Shared domain complication for triangle inequality:

Condition	$T(p,q), T(q,r), T(p,r)$ relationship	Triangle inequality	Resolution
Common ground	All computed over $T^* = \cap T(p_i, p_j)$	Holds automatically	Standard metric space
Nested domains	$T(p,r) \subseteq T(p,q) \cap T(q,r)$	Holds: fewer terms on left than sum on right	Safe for pairwise approach

Condition	$T(p,q), T(q,r), T(p,r)$ relationship	Triangle inequality	Resolution
Disjoint domains	$T(p,q) \cap T(q,r) \cap T(p,r) = \emptyset$	Undefined: no common basis for comparison	No meaningful distance; processors are in different fields
Partially overlapping	Some but not all tasks shared across all three pairs	May fail: distances measured on different task subsets are incommensurable	Restrict to common ground or accept pseudo-metric

Table C: Distance Function Comparison

Property	L^2 (Euclidean)	L^1 (Manhattan)	L^∞ (Chebyshev)	Weighted L^2
What it measures	Overall profile similarity with outlier sensitivity	Total op count difference across all tasks	Maximum single-task difference	Operationally weighted similarity
Physical interpretation	Straight-line distance in task space	Sum of all per-task cost differences	Worst-case skill gap on any single task	Distance weighted by task importance
Sensitive to	Large differences on individual tasks (squared)	Uniform to all differences regardless of magnitude	Only the single largest difference	Differences on high-weight tasks
Best for	General similarity assessment; clustering	Total cost comparison; resource estimation	Bottleneck identification; certification	Operational staffing; risk assessment
When two processors differ by 10 ops on 1 task vs 1 op on 100 tasks	Equal: $\sqrt{100} = \sqrt{100} = 10$	Different: 10 vs 100; Manhattan says 100-task difference is $10 \times$ larger	Different: 10 vs 1; Chebyshev says single-task difference is $10 \times$ larger	Depends on weights

Property	L^2 (Euclidean)	L^1 (Manhattan)	L^∞ (Chebyshev)	Weighted L^2
Cluster shape tendency	Spherical clusters	Diamond-shaped clusters	Cubic clusters	Ellipsoidal clusters aligned with weights
Computational cost	$O(m)$ per pair	$O(m)$ per pair	$O(m)$ per pair	$O(m)$ per pair
Sensitivity to dimensionality	Distances grow with $\sqrt{m}$; normalize by $\sqrt{m}$ for cross-set comparison	Distances grow with m ; normalize by m	No growth with dimensionality; always in single-task units	Growth depends on weight distribution

Table D: Profile Types and Geometric Signatures

Profile Type	Distance from Origin	Angular Distribution	Cluster Membership	Example	Geometric Signature
Complete novice	Maximum (far)	Uniform across axes; no task dissolved	Novice shell, no specialization wedge	First-day medical student; new hire on unfamiliar code-base	Point in far corner of positive orthant; all components high
Developing generalist	Moderate	Broadly distributed; many tasks partially dissolved	Intermediate shell, broad wedge	Third-year resident; mid-career generalist developer	Point moving toward origin along many axes simultaneously

Profile Type	Distance from Origin	Angular Distribution	Cluster Membership	Example	Geometric Signature
Developing specialist	Moderate overall; near origin on specialty axes	Concentrated; specialty tasks dissolved, others remain high	Intermediate shell, narrow wedge	Cardiology fellow; back-end specialist developer	Point near origin in some dimensions, far in others; elongated profile
Expert generalist	Near origin	Broadly distributed near origin; most tasks dissolved	Expert shell, broad wedge	Attending with decades of varied practice; senior full-stack developer	Point near origin along most axes; few high-cost outliers
Expert specialist	Near origin on specialty axes; moderate to far on others	Concentrated near origin on specialty, spread on rest	Expert shell on specialty slice; intermediate on others	World-class cardiac surgeon who hasn't done general surgery in years	Very near origin in specialty dimensions; moderate elsewhere

Profile Type	Distance from Origin	Angular Distribution	Cluster Membership	Example	Geometric Signature
Degraded expert	Moderate (regression from near-origin)	Previously concentrated near origin; some axes moved outward	Moving from expert shell toward intermediate	Surgeon re-turning after long absence; development on unfamiliar new stack	Point that was near origin, now further along specific axes; cascade signature
Polymath	Near origin across multiple specialization wedges	Broad coverage; dissolved tasks spanning multiple conventional specialties	Member of multiple specialty clusters simultaneously	Emergency physician (broad dissolution); senior site reliability engineer (full-stack dissolution)	Point near origin in multiple orthogonal dimensions
Savant	Extreme asymmetry; zero on some axes, maximum on others	Maximally concentrated	Extreme specialist cluster; outlier from all generalist clusters	World-class narrow specialist with no breadth	Point at origin on few axes, far corner on rest; maximum profile asymmetry

Table E: Skill Gap Decomposition

Gap Shape	Profile	Component Distribution	Magnitude	Training Prescription	Detection Method
Concentrated spike	Few large components, most near zero	1–3 tasks account for >80% of gap magnitude	Low to moderate	Targeted: dissolve the specific tasks with large gap components	Sort gap components descending; if top 3 account for majority, gap is concentrated
Distributed flat	Many moderate components, no extremes	No single task accounts for >10% of gap magnitude	Moderate to high	General: broad experience across task set; no single task is the problem	Compute coefficient of variation of gap components; low CV = distributed
Bimodal	Two clusters of gap components: near-zero and moderate-to-high	Two distinct groups visible in sorted profile	Moderate	Hybrid: the near-zero tasks are dissolved; the moderate cluster is a coherent skill area needing development	Histogram of gap components shows two modes

Gap Shape	Profile	Component Distribution	Magnitude	Training Prescription	Detection Method
Staircase	Graduated components from small to large with no sharp jump	Components evenly spread across range	Moderate to high	Sequential: tasks can be prioritized by gap size; dissolve largest gaps first for maximum closure rate	Sorted gap components show linear rather than exponential shape
Single outlier	One extreme component, all others near zero	One task accounts for >90% of gap magnitude	Low (driven by one task)	Highly targeted: one specific task requires attention	Maximum component » mean of remaining components
Inverted (strength profile)	Negative components dominate; processor is cheaper than reference on most tasks	Most components negative; processor outperforms reference	Negative net gap	Not a gap but a strength profile; reference may learn from this processor	Sum of components is negative; processor-reference direction is reversed

Table F: Processing Entropy Matrix Structure

Matrix Property	Definition	When Expected	Implication	Measurement
Low rank ($k \ll \min(n,m)$)	Matrix well-approximated by rank-k factorization	Processors share training tradition; tasks share structural patterns	Few underlying skill factors explain most variation; profile space has low effective dimensionality	Singular value decomposition; count singular values above noise threshold
Full rank ($k \approx \min(n,m)$)	No low-rank approximation adequate	Processors are highly heterogeneous; tasks are structurally independent	Each task measures something unique; no compression of profile space possible	SVD shows gradual singular value decay with no clear cutoff
Block diagonal	Matrix decomposes into independent blocks	Processor set contains distinct subpopulations with non-overlapping task domains	Subpopulations are genuinely in different fields; cross-block comparison meaningless	Permute rows and columns to reveal block structure; off-diagonal blocks near-zero or undefined
Banded	Entries cluster near diagonal after appropriate ordering	Tasks and processors can be ordered along a single difficulty/skill continuum	Simple linear skill progression; one-dimensional expertise	Sort rows by total profile magnitude, columns by mean cost; check for band structure
Sparse	Many zero entries (dissolved tasks)	Mature processor population; most routine dissolved	High expertise; focus on remaining non-zero entries for further development	Count zero vs non-zero entries; ratio is population dissolution maturity
Dense with few zeros	Few zero entries	Immature population or extremely broad task set	Low dissolution; most tasks still cost ops	Same count; ratio indicates development stage

Singular value decay patterns and interpretations:

Decay Pattern	Shape	Interpretation	Typical Domain
Sharp elbow	First k values large, then sudden drop	k clear skill factors; clean low-rank structure	Structured training program with defined competency areas
Gradual decay	Singular values decrease smoothly with no elbow	No clear factor structure; continuous spectrum of skill dimensions	Self-directed learning; highly varied task domain
Two elbows	Drop, plateau, second drop	Two scales of structure: broad skill factors and fine-grained specializations	Large domain with both general competencies and subspecialties
One dominant	First singular value » rest	One factor (overall experience) explains most variation; novice-to-expert is primarily one-dimensional	Domains where skill is primarily about hours of practice
Two comparable, then drop	First two values large and similar, then drop	Two independent skill dimensions of roughly equal importance	Domains with two distinct competency areas (e.g., diagnostic skill vs procedural skill)

Table G: Trajectory Properties

Property	Definition	Formula	Interpretation	Healthy Range
Dissolution velocity magnitude	Total rate of profile change	$ \mathbf{v}(p,t) = \sqrt{(\sum_i (dH_p(t_i)/dt)^2)}$	High = rapid skill change (improvement or degradation); low = plateau or stability	Domain-dependent; should be positive during training, near-zero at expertise

Property	Definition	Formula	Interpretation	Healthy Range
Dissolution velocity direction	Which tasks are changing fastest	$\mathbf{v}(\mathbf{p}, t) / \mathbf{v}(\mathbf{p}, t) $ (unit vector)	Points toward axes where dissolution is most active	Should align with training priorities; misalignment = inefficient training
Trajectory efficiency	Displacement / path length	$\eta = \mathbf{H}(\mathbf{p}, t_k) - \mathbf{H}(\mathbf{p}, t_0) / \sum_j \mathbf{H}(\mathbf{p}, t_{j+1}) - \mathbf{H}(\mathbf{p}, t_j) $	Near 1 = direct progress; low = cascades, backtracking, oscillation	0.7–1.0 for stable training; <0.5 suggests frequent disruption
Mean component velocity	Average dissolution rate per task	$\bar{v} = (1/m) \sum_i d\mathbf{H}_p(t_i)/dt$	Negative = net dissolution in progress; zero = plateau; positive = net regression	Should be negative during active training
Velocity variance	How unevenly dissolution is distributed across tasks	$\text{var}(d\mathbf{H}_p(t_1)/dt, \dots, d\mathbf{H}_p(t_m)/dt)$	High = concentrated dissolution (specialist trajectory); low = uniform dissolution (generalist trajectory)	Neither inherently better; reflects training strategy
Cascade signature	Sudden positive velocity spike followed by recovery	Sharp positive excursion in $ \mathbf{v} $ with positive mean component velocity, followed by return to negative	Cascade event promoted dissolved elements; recovery is restabilization	Infrequent and brief in robust expertise; frequent and prolonged indicates fragility
Plateau detection	Velocity magnitude near zero for extended period	$ \mathbf{v}(\mathbf{p}, t) < \epsilon$ for $t > t_{\text{threshold}}$	Processor has stopped improving; either fully dissolved (expert plateau) or stalled (development plateau)	Expert plateau: desirable. Development plateau: intervention needed

Property	Definition	Formula	Interpretation	Healthy Range
Convergence rate to reference	Rate of approach to reference profile	$-d(d(p,r))/dt$ for reference processor r	Positive = closing gap; negative = falling behind; zero = stable gap	Should be positive during training; converges to zero as gap closes

Table H: Cluster Types and Detection

Cluster Type	Structure	Detection Method	Interpretation	Example
Skill level (radial)	Concentric shells at different distances from origin	Cluster by $d_2(p, \text{origin})$; k-means with k = expected skill levels	Processors at similar total dissolution levels	Novice / intermediate / expert shells in any domain
Specialization (angular)	Wedges at similar distance from origin but different directions	Normalize profiles to unit length; cluster normalized vectors	Processors with similar dissolution patterns but possibly different amounts	Cardiac vs neuro vs general surgery at same experience level
Combined (radial + angular)	Shells subdivided into wedges	Two-stage clustering: first by distance from origin, then within each shell by direction	Full skill landscape: both level and specialty	Subspecialty-specific expertise tiers
Transition zone	Sparsely populated region between clusters	Density-based methods (DBSCAN); identify low-density boundaries	Skill levels that processors pass through quickly; unstable intermediate states	Rapid transition from supervised to independent practice
Attractor	Dense point in profile space; many trajectories converge toward it	Identify high-density regions that trajectories approach	Natural resting states of expertise; positions where dissolution and demand reach equilibrium	Experienced generalist profile in stable practice

Cluster Type	Structure	Detection Method	Interpretation	Example
Bifurcation point	Point where trajectories diverge into different specialization clusters	Identify profile positions where trajectory variance is maximally high	Decision points where processors commit to different specialization paths	Point in residency where sub-specialty training begins
Outlier	Processor far from all clusters	Distance to nearest cluster centroid exceeds threshold	Unusual profile: exceptional, narrow, degraded, or from different training tradition	Self-taught developer; physician from foreign training system

Table I: Task Topology Properties

Property	Definition	Computation	Interpretation	Application
Task cluster	Group of tasks with small pairwise distance	Standard clustering on task distance matrix	Tasks that all processors find similarly costly; natural difficulty classes	Curriculum design: cluster-aligned modules
Discriminating task	Task with high cost variance across processors	$\text{disc}(t) = \text{var}(\text{Hp}_1(t), \dots, \text{Hp}_n(t))$	Task that effectively separates skill levels; processors diverge most on this task	Assessment design: include high-discrimination tasks
Universal easy	Task near origin in task-cost space (all processors low cost)	Mean cost near zero; low variance	Task that virtually everyone has dissolved; low assessment value	Exclude from assessment; low training priority

Property	Definition	Computation	Interpretation	Application
Universal hard	Task far from origin (all processors high cost)	Mean cost high; low variance	Task that no one has dissolved; may indicate domain boundary or genuinely difficult element	Research target: why hasn't this dissolved?
Transfer neighborhood	Set of tasks connected by high transfer affinity	Transfer affinity graph: edges above threshold	Dissolving one task in neighborhood accelerates others; curriculum leverage point	Training sequencing: dissolve high-leverage tasks first
Bridge task	Task connecting two otherwise-distant task clusters	High betweenness centrality in task graph	Task that, if dissolved, enables transfer between otherwise-independent skill areas	Strategic training target: bridges unlock cross-domain skill transfer
Isolated task	Task far from all others in task distance	No edges above threshold in task graph	Task requiring unique processing chain with no structural similarity to others	Must be trained independently; no transfer benefit from other tasks
Hub task	Task close to many others; central in task graph	High degree centrality in task graph	Task whose processing chain shares structure with many others; maximum transfer potential	Highest-leverage training target; dissolving hub accelerates many neighbors

Table J: Domain-Specific Measurement Specifications

Domain	Task Set Source	Processor Set	Op Counting Method	Processing Entropy Proxy	Profile Update Frequency	Typical Matrix Size
Medicine (emergency)	ICD-coded presentations; chief complaint categories; clinical vignettes	Physicians at various training stages	Direct observation; video review; chart times-tamps; simulation performance	Time to disposition; diagnostic accuracy composite	Per shift or per rotation	20–50 processors × 50–200 task types
Software engineering	Bug taxonomy by root cause; feature types by complexity class; code review categories	Development team members	IDE telemetry; VCS times-tamps; screen recording; PR review duration	Time from assignment to resolution; lines of debugging before fix	Weekly or per sprint	5–30 processors × 30–100 task types
Combat aviation	Threat scenarios by geometry and type; BFM setups; intercept profiles; emergency procedures	Pilots at various qualification levels	HUD tape; flight recorder; debrief scoring; simulator instrumentation	Time to valid weapons solution; kill-to-loss ratio; checklist completion time	Per sortie or per training block	10–40 processors × 40–80 task types

Domain	Task Set Source	Processor Set	Op Counting Method	Processing Entropy Proxy	Profile Update Frequency	Typical Matrix Size
Computational (cache)	Memory access patterns: sequen- tial, strided (vari- ous), random (various locality), pointer- chasing (various depth)	Cache configu- rations: associa- tivity $\times$ capacity $\times$ replace- ment policy $\times$ prefetch strategy	Hardware perfor- mance counters: cache- misses, cache- references, cycles per access	Average memory access time in cycles	Per benchmark run	10–50 configura- tions $\times$ 20–60 patterns
Manufacturing	Assembly opera- tions by product variant and station; defect types by root cause	Line workers at various experi- ence levels	Time-and- motion study; video with motion capture; defect rate per operation	Cycle time per operation; defect rate	Daily or weekly	10–40 processors $\times$ 20–80 operations
Customer support	Ticket cate- gories by issue type and com- plexity; cus- tomer interac- tion scenar- ios	Support agents at various tiers	Ticketing system times- tamps; call record- ings; resolution codes	Time to resolution; escalation rate; first-contact resolution rate	Weekly or monthly	10–30 processors $\times$ 20–60 ticket types

Domain	Task Set Source	Processor Set	Op Counting Method	Processing Entropy Proxy	Profile Update Frequency	Typical Matrix Size
Air traf- fic con- trol	Traffic scenar- ios by density, geome- try, weather, equip- ment status	Controllers at various rating levels	Audio record- ings; eye tracking; STARS/ERAM system logs; simulation scoring	Time to conflict resolution; operational errors per session; com- munication efficiency	Per session or per scenario set	10–30 processors × 30–70 scenarios
Music per- for- mance	Repertoire pieces by tech- nical demand; sight- reading passages by com- plexity; ensem- ble scenar- ios	Musicians at various skill levels	Audio/video analysis; perfor- mance scoring; practice log analysis	Error rate; timing accuracy; expression fidelity	Per practice session or per performance cycle	5–20 processors × 30–100 pieces
Cooking (pro- fes- sional)	Dishes by tech- nique category and com- plexity; knife skills; plating; timing manage- ment	Kitchen staff at various experi- ence levels	Video analysis; ticket time stamps; tasting panel scores	Time per dish; defect rate; consistency across repetitions	Per service or per week	5–20 processors × 20–60 dish types

Domain	Task Set Source	Processor Set	Op Counting Method	Processing Entropy Proxy	Profile Update Frequency	Typical Matrix Size
Mathematics education	Problem types by topic and diffi- culty; proof tech- niques; compu- tation methods	Students at various course levels	Written work (line count as op proxy); exam timing; homework comple- tion time	Time per problem; error rate; solution elegance (step count)	Per assignment or per exam	20–50 processors × 30–80 problem types

Table K: Transfer Affinity Properties

Property	Definition	Measurement	Implication
Symmetric affinity	$\text{aff}(t_i, t_j) = \text{aff}(t_j, t_i)$	Dissolving either task equally benefits the other	Shared processing chain structure; order of dissolution doesn't matter
Asymmetric affinity	$\text{aff}(t_i, t_j) \neq \text{aff}(t_j, t_i)$	Dissolving t_i helps t_j more than the reverse	t_i contains t_j 's processing chain as a sub-chain but not vice versa; t_i is the more general skill
Zero affinity	$\text{aff}(t_i, t_j) = 0$	Dissolving either has no effect on the other	Independent processing chains; no shared structure
Negative affinity	Dissolving t_i increases first-encounter cost of t_j	Rare; possible when dissolution of t_i creates assumptions that interfere with t_j	Conflicting processing chains; specialist dissolution can narrow generalist capability
Transitive affinity	$\text{aff}(t_i, t_j) > 0$ and $\text{aff}(t_j, t_k) > 0$ implies $\text{aff}(t_i, t_k) > 0$	Tasks form transfer chains; dissolving one end benefits the far end through intermediaries	Connected transfer neighborhoods; curriculum can exploit transitivity

Property	Definition	Measurement	Implication
Non-transitive affinity	$\text{aff}(t_i, t_j) > 0$ and $\text{aff}(t_j, t_k) > 0$ but $\text{aff}(t_i, t_k) = 0$	Intermediate task shares structure with both endpoints but endpoints share nothing with each other	Hub tasks in transfer graph; t_j is a bridge between unrelated skill areas
Decay with distance	Transfer affinity decreases with processing entropy distance between tasks	Fit affinity vs distance; test correlation	Nearby tasks in processing entropy space tend to share structure; distance is a (noisy) proxy for transfer
Independence from distance	Transfer affinity uncorrelated with processing entropy distance	Same fit shows no correlation	Similar cost does not imply shared structure; distance and transfer measure different properties

Table L: Computational Test Case — Software Engineering

Metric	Example Measurement	Source	Interpretation
Processing entropy per developer-bug pair	Developer A: null pointer bug = 4 ops (read stacktrace, identify cause, fix, verify). Developer B: same bug = 18 ops (reproduce, add logging, search codebase, identify pattern, research, try fix, fail, try second fix, verify)	IDE telemetry + VCS timestamps	A has dissolved null pointer debugging for this codebase; B has not
Processor distance $d_2(A, B)$	$\sqrt{(4^2 + 14^2 + 2^2 + \dots)}$ across 40 bug types = 47.3 ops	Computed from matrix	A and B differ by 47.3 ops (Euclidean) across the full bug taxonomy
Gap profile shape for B relative to team senior	Concentrated: 3 bug types account for 78% of gap magnitude	Sorted gap components	B needs targeted training on specific bug categories, not general experience
Task cluster: “configuration bugs”	Config parsing, environment variable, deployment config, feature flag bugs cluster at distance < 5	Task distance matrix clustering	These bugs require similar debugging chains regardless of specific config system

Metric	Example Measurement	Source	Interpretation
Discriminating task	Race condition bugs: variance = 182 ops ² across 12 developers	var of cost column	Race condition debugging is the best single predictor of developer seniority on this codebase
Hub task (transfer)	Understanding the build system: high transfer affinity to 8 other bug types	Transfer affinity graph degree centrality	New developers should learn the build system early; it accelerates debugging across many categories
Trajectory: new developer months 1–6	Month 1: d_2 to team median = 89. Month 3: 54. Month 6: 31	Monthly profile computation	Healthy dissolution trajectory; gap closing at decreasing rate (consistent with power law)
Cascade signature	Sprint 4: codebase refactor; developer C’s profile distance from origin increased by 23 ops across 6 bug types, recovered over 3 weeks	Profile tracked weekly	Refactor invalidated dissolved navigation patterns; cascade of 6 promotions; recovery took 15+ working days
Matrix rank	40 bug types $\times$ 12 developers; effective rank = 4 (first 4 singular values capture 87% of variance)	SVD of processing entropy matrix	Four underlying skill dimensions explain most of the variation in this team’s bug-fixing capability
Skill factors (from factorization)	Factor 1: system architecture understanding. Factor 2: debugging tool proficiency. Factor 3: language-specific knowledge. Factor 4: domain/business logic	Interpretation of SVD factors by examining task loadings	Four trainable competency areas; each developer’s position on these four dimensions summarizes their profile

Table M: Computational Test Case — Cache Behavior

Metric	Example Measurement	Source	Interpretation
Processing entropy per config-pattern pair	4-way L1 32KB, sequential scan = 4.2 cycles/access. Same config, random access = 187 cycles/access	Hardware performance counters	Sequential access fully dissolved (cached); random access near main memory cost (not cached)

Metric	Example Measurement	Source	Interpretation
Configuration distance $d_2(C_1, C_2)$	4-way 32KB L1 vs 8-way 64KB L1: $d_2 = 34.7$ cycles across 30 access patterns	Computed from matrix	Configurations differ by 34.7 cycles on average across pattern set; larger cache is uniformly better but not by equal amount on all patterns
Task distance $d_2(\text{sequential, random})$	$d_2 = 412$ cycles across 20 configurations	Task distance matrix	Every configuration finds these patterns dramatically different; they are far apart in task space
Task cluster: “locality-dependent patterns”	Sequential, strided-4, strided-8, and small-working-set random cluster together	Task distance clustering	These patterns all benefit primarily from spatial locality; cache line prefetch helps all equally
Discriminating pattern	Pointer-chasing with depth 6: variance = 8,400 cycles ² across configurations	Variance of cost column	This pattern most discriminates between cache configurations; large vs small cache diverge maximally here
Transfer affinity	Sequential → strided-4: high (0.87). Sequential → random: low (0.12)	Measure cost reduction of pattern B when cache is warm from pattern A	Prefetch strategy that helps sequential also helps strided; does not help random
Matrix rank	20 configurations × 30 patterns; effective rank = 3	SVD	Three underlying dimensions: capacity tier, associativity class, prefetch aggressiveness
Cascade: context switch cost	Process A warm cache → context switch → process B runs → switch back: 847 cache misses at ~180 cycles each = ~152K cycles total	perf stat before/after context switch	Process A’s dissolution inventory (cached entries) had 847 elements promoted; recovery cost is the cascade cost; dwarfs register save/restore (~200 cycles)

Table N: Cross-Domain Profile Structure Comparison

Domain (k)	Typical Effective Dimensionality	Dominant Factor	Second Factor	Typical Skill Level Clusters	Specialization Clusters	Novice-to-Expert Trajectory Shape
Emergency medicine	4-6	Pattern recognition speed	Procedural fluency	1 (student, junior resident, senior resident, attending)	3-5 (trauma, cardiac, pediatric, toxicology, general)	Power-law approach along all axes; specialty divergence in year 3-4
Software engineering (single code-base)	3-5	System architecture understanding	Debugging proficiency	2 (junior, mid, senior)	2-4 (frontend, backend, infrastructure, data)	Rapid initial dissolution of common bugs; long tail on rare/complex bugs
Combat aviation	3-4	Threat assessment speed	Weapons employment accuracy	3-4 (student, wingman, flight lead, instructor)	2-3 (air-to-air, air-to-ground, electronic warfare)	Step function at qualification milestones; continuous refinement between
Manufacturing	2-3	Motor execution speed	Quality discrimination	3 (trainee, qualified, expert)	By product line or station	Exponential-like; motor skills dissolve faster than quality judgment
Customer support	2-3	Issue classification speed	System navigation fluency	2-3 (new, experienced, specialist)	By product area	Rapid initial dissolution; plateau at product boundary; second curve on escalations

Domain (k)	Typical Effective Dimensionality	Dominant Factor	Second Factor	Typical Skill Level Clusters	Specialization Clusters	Novice-to-Expert Trajectory Shape
Cooking (professional)	3–4	Technique execution	Timing management	3 (apprentice, cook, chef)	By station (sauté, grill, pastry, garde manger)	Power-law with station-change cascades
Air traffic control	3–5	Spatial pattern recognition	Communication efficiency	3–4 (student, developmental, certified, expert)	By sector type (terminal, en-route, approach)	Step function at certification; continuous refinement of efficiency
Mathematics education	2–4	Symbolic manipulation fluency	Proof strategy selection	4–5 (by course level)	By subfield (algebra, analysis, geometry, combinatorics)	Power-law per topic; plateau between topic transitions

Table O: Specification Summary

Metric	Count
Formal definitions	17
Distance functions defined	4 (L^2 , L^1 , L^∞ , weighted)
Metric axioms verified	4 (non-negativity, identity, symmetry, triangle inequality)
Shared domain resolution approaches	2 (common-ground, pairwise)
Profile types characterized	8
Gap shape categories	6
Matrix structure types	6
Singular value decay patterns	5
Trajectory properties defined	8
Cluster types	7
Task topology properties	8
Transfer affinity properties	8
Domain measurement specifications	10

Metric	Count
Computational test cases	2 (software engineering, cache behavior)
Cross-domain structure comparisons	8
Open problems	6

Key Equations Summary

Processing entropy profile:

$$\mathbf{H}(p) = (Hp(t_1), Hp(t_2), \dots, Hp(t_m))$$

Shared domain:

$$T(p, q) = \{ t \in T : Hp(t) \text{ defined} \wedge Hq(t) \text{ defined} \}$$

Euclidean distance between processors:

$$d_2(p, q) = \sqrt{(\sum_i (Hp(t_i) - Hq(t_i))^2)}$$

Manhattan distance between processors:

$$d_1(p, q) = \sum_i |Hp(t_i) - Hq(t_i)|$$

Chebyshev distance between processors:

$$d_\infty(p, q) = \max_i |Hp(t_i) - Hq(t_i)|$$

Weighted distance between processors:

$$d_w(p, q) = \sqrt{(\sum_i w_i (Hp(t_i) - Hq(t_i))^2)}$$

Task discrimination power:

$$\text{disc}(t) = \text{var}(Hp_1(t), Hp_2(t), \dots, Hp_n(t))$$

Skill gap vector:

$$\mathbf{g}(p, r) = \mathbf{H}(p) - \mathbf{H}(r)$$

Gap closure rate:

$$\rho(p, r, t) = -d|\mathbf{g}(p, r)|/dt$$

Dissolution velocity:

$$\mathbf{v}(p, t) = d\mathbf{H}(p)/dt$$

Trajectory efficiency:

$$\eta = |\mathbf{H}(p, t_k) - \mathbf{H}(p, t_0)| / \sum_j |\mathbf{H}(p, t_{j+1}) - \mathbf{H}(p, t_j)|$$

Matrix factorization:

$$H \approx U(n \times k) \times V(k \times m)$$

Throughput (from prior work, referenced throughout):

$$\text{Throughput} = N / (\bar{d} \times \bar{H}_p)$$

Fundamental inequality (from prior work, referenced throughout):

$$\Sigma \text{ ops} \times \bar{d} \leq N$$

[[HOWL-MATH-17-2026](#)] Processing Entropy as a Metric Space. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-17-2026>

[[HOWL-INFO-11-2026](#)] The Relationship of Zero, One, and Infinity in Information Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-11-2026>

[[HOWL-INFO-12-2026](#)] Information Processing Requires Reduction to Cardinality One. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-12-2026>

[[HOWL-INFO-13-2026](#)] The Six States of Information. Github: <https://github.com/ghowland/papers/tree/main/papers/INFO-13-2026>

[[HOWL-MATH-14-2026](#)] A Mathematical Theory of Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-14-2026>

[[HOWL-MATH-15-2026](#)] A Measurement Theory of Processing. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-15-2026>

[[HOWL-MATH-16-2026](#)] The Geometry of Dissolution and Fragility. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-16-2026>